University of Sunderland

Faculty: Programme

NAME: Sugal Darlami

MODULE CODE: CET 256

MODULE TITLE: Cyber Security

MODULE LEADER: Matthew Banton

SUBMISSION DATE AND TIME: 16 oct

Canvas id: bj00oa

ASSIGNMENT: 2

Academic Misconduct is an offence under university regulations, and this involves:

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:

• I have used artificial intelligence tools to generate an idea for my assignment:	**NO**
• I have used artificial intelligence tools to write my assignment for me:	**NO**
• I have used artificial intelligence tools to brainstorm ideas for my assignment:	**NO**
• I have used artificial intelligence tools to correct my original assignment:	**NO**

DECLARATION

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.
- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

1. Executive Summary

This report presents a security audit of a WordPress application hosted within a local virtual honeypot environment. The assessment was conducted using the industry-standard security scanner WPScan v3.8.28. A non-intrusive black-box assessment was performed to evaluate the application's server configuration, exposed endpoints, and potential security weaknesses.

The assessment identified five significant security misconfigurations and attack vectors:

1. XML-RPC Interface Enabled (/xmlrpc.php): The XML-RPC interface may increase exposure to automated brute-force authentication attempts and Pingback Distributed Denial of Service (DDoS) attacks.
2. Directory Browsing Enabled (/wp-content/uploads/): Directory listing allows unauthenticated users to browse uploaded files, potentially resulting in information disclosure.
3. HTTP Server Header Disclosure: HTTP response headers reveal server and software information, including Apache/2.4.58, PHP/8.2.12, and Win64. This information could assist attackers in identifying suitable vulnerabilities and exploits.
4. Active External WP-Cron (/wp-cron.php): WordPress scheduled tasks are triggered through web requests, which may contribute to resource exhaustion and affect system availability under excessive request conditions.
5. Username Enumeration via REST API (cyber_admin): The REST API exposes a valid administrative username, providing attackers with useful information for targeted authentication attacks.

To mitigate these risks without modifying the live honeypot environment, several security-hardening controls are recommended. These include disabling XML-RPC where it is not required, preventing directory indexing, reducing unnecessary server-information disclosure, configuring server-side cron jobs, and restricting public access to REST API user information.

Overall, implementing these controls would strengthen the application's security posture across the Confidentiality, Integrity, and Availability (CIA) Triad, while reducing unnecessary information exposure and potential attack surfaces.

Table of Contents

1. Executive Summary……...……..2

2. Introduction……..4

3. Identified Vulnerability……...5

4. Identified Security Fixes……6

3.1 Disable XML-RPC ……...6

3.2 Disable Directory Indexing ……..6

3.3 Hide Server Information……6

3.4 Configure System Cron……7

3.5 Restrict User Enumeration ……...7

5. Conclusion……7

6. References……8

7. Appendix ……9

7.1 Figure 7.1: WPScan Initial Scan 9

7.2 Figure 7.2: WPScan User Enumeration 10

7.3 Raw WPScan Output 11

2. Introduction

WordPress runs a substantial part of the world's websites, making it an common target for automated cyber-attacks. In this regard, performing regular security assessments of Web applications is essential to ensure that they are not subject to unauthorized access, data breaches, and service disruption.

This paper contains the results of a security audit of a WordPress blog hosted on a virtual honeypot located at http://192.168.1.71/wordpress-6.5-en_GB/wordpress/. The scan was performed with the open-source vulnerability scanner for WordPress called WPScan version 3.8.28 that implements a black-box approach to probe the target domain for security weaknesses. The main objectives were to identify potential vulnerabilities and security misconfigurations, assess their potential impact using the Confidentiality, Integrity and Availability (CIA) Triad, and propose appropriate remediation strategies.

3. Identified Vulnerability

The automated scan identified several security weaknesses and misconfigurations in the target application.

ID	Vulnerability / Finding	Evidence	Potential Impact	CIA Triad
A	XML-RPC Interface Enabled (/xmlrpc.php)	WPScan confirmed that xmlrpc.php is accessible on the target (WPScan, 2026).	XML-RPC can be abused for automated brute-force authentication attempts using system.multicall and for Pingback-based DDoS attacks. This may result in credential compromise and increased server resource consumption.	Confidentiality, Availability
B	Directory Browsing Enabled (/wp-content/uploads/)	Directory listing was accessible within the WordPress uploads directory (WPScan, 2026).	Unauthenticated users may view and download uploaded files. If sensitive documents or assets are stored in the directory, this can lead to information disclosure.	Confidentiality
C	Server Information Disclosure (HTTP Headers)	HTTP response headers revealed Apache/2.4.58 (Win64), OpenSSL/3.1.3, and PHP/8.2.12 (WPScan, 2026).	Exposing software and version information can help attackers identify known vulnerabilities and select targeted exploitation techniques (OWASP, 2021; PHP Manual, 2024) .	Confidentiality
D	External WP-Cron Enabled (/wp-cron.php)	WP-Cron was observed to execute through normal page requests (WPScan, 2026).	Automated requests to wp-cron.php could consume server resources and potentially contribute to performance degradation or denial-of-service conditions (Rapid7, 2024; Stallings, 2018).	Availability
E	User Enumeration via REST API (cyber_admin)	WPScan identified the valid username cyber_admin through /wp-json/wp/v2/users/ (WPScan, 2026; WordPress Developer Resources, 2024)..	Disclosure of valid usernames makes password-guessing and brute-force attacks easier. Successful account compromise could allow unauthorized access and modification of website content (NIST, 2020; Zamościński and Kozieł, 2020).	Confidentiality, Integrity

4. Identified Security Fixes

The vulnerabilities found during the security assessment can be fixed by applying several security measures. Since this system is being used as a honeypot, these fixes are only recommendations and were not applied to the system.

4.1 Disable XML-RPC

XML-RPC should be disabled because it can be used for brute-force attacks and pingback abuse. It can be disabled in the WordPress functions.php file using the following code:

add_filter('xmlrpc_enabled', '__return_false');

Another option is to block access to xmlrpc.php using Apache .htaccess rules (WordPress Documentation, 2024).

4.2 Disable Directory Indexing

Directory listing should be disabled so that visitors cannot view the files stored inside a directory. Apache can prevent directory listing by adding the following setting to the .htaccess file:

Options -Indexes

The same setting can also be configured in Apache's httpd.conf file (Apache Software Foundation, 2024; OWASP, 2021).

4.3 Hide Server Information

The server currently reveals information about the software being used. This information can help an attacker identify possible vulnerabilities. Apache can be configured with:

ServerTokens Prod

ServerSignature Off

PHP version information can also be hidden by setting the following option in php.ini:

expose_php = Off

These settings reduce the amount of technical information exposed to someone scanning the website (Apache Software Foundation, 2024; PHP Manual, 2024).

.

4.4 Configure System Cron

WordPress uses wp-cron.php to run scheduled tasks. Because it can run when the website receives requests, it may use additional server resources. A better approach is to disable WordPress's built-in cron and use the operating system's scheduled tasks instead.

This can be done in wp-config.php using:

define('DISABLE_WP_CRON', true);

A system cron job can then be created to run the required WordPress tasks at regular intervals (Rapid7, 2024).

4.5 Restrict User Enumeration

User enumeration should be restricted because it can reveal usernames that attackers may use in further login attacks. One endpoint that should be restricted is:

/wp-json/wp/v2/users

Access to this endpoint can be restricted using WordPress security settings, security plugins, or web-server rules. This prevents unnecessary user information from being publicly available (WordPress Developer Resources, 2024).

5. Conclusion

The security audit showed that the default WordPress configuration can expose the website to several security risks. The assessment identified issues such as enabled XML-RPC, user enumeration through the REST API, directory listing, and server information disclosure. These issues could help attackers gather information about the system or carry out further attacks.

The recommended security fixes would reduce these risks by limiting unnecessary access and hiding sensitive server information. Overall, the audit shows the importance of reviewing default configurations and applying basic security controls when deploying a web application.

6. References

Apache Software Foundation, 2024. *Core - Apache HTTP Server Version 2.4*. Available at: https://httpd.apache.org/docs/2.4/mod/core.html [Accessed 1 October 2026].

National Institute of Standards and Technology (NIST), 2020. *Security and Privacy Controls for Information Systems and Organizations: NIST Special Publication 800-53 Revision 5*. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://csrc.nist.gov/pubs/sp/800/53/r5/final [Accessed 1 October 2026].

OWASP, 2021. *OWASP Top 10:2021*. Open Web Application Security Project. Available at: https://owasp.org/Top10/ [Accessed 1 October 2026].

PHP Documentation Group, 2024. *Description of core php.ini directives: expose_php*. PHP Manual. Available at: https://www.php.net/manual/en/ini.core.php#ini.expose-php [Accessed 2 October 2026].

Rapid7, 2024. *Wordpress XMLRPC DoS*. Rapid7 Vulnerability & Exploit Database. Available at: https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/ [Accessed 2 October 2026].

Stallings, W., 2018. *Effective Cybersecurity: A Guide to Using Best Practices and Standards*. Boston: Addison-Wesley Professional. Available at: https://www.pearson.com/en-gb/subject-catalog/p/effective-cybersecurity-a-guide-to-using-best-practices-and-standards/P200000007404 [Accessed 3 October 2026].

WordPress, 2024. *XML-RPC Support*. WordPress.org. Available at: https://codex.wordpress.org/XML-RPC_Support [Accessed 1 October 2026].

WordPress Developer Resources, 2024. *Users – REST API Handbook*. WordPress.org. Available at: https://developer.wordpress.org/rest-api/reference/users/ [Accessed 4 October 2026].

WPScan, 2026. *WPScan WordPress Vulnerability Database and Security Scanner*. Available at: https://wpscan.com/ [Accessed 4 October 2026].

Zamościński, P. and Kozieł, G., 2020. 'Analysis of security CMS platforms by vulnerability scanners', *Journal of Computer Sciences Institute*, 16, pp. 261–268. Available at: https://doi.org/10.35784/jcsi.2020 [Accessed 5 October 2026].

7. Appendix: WPScan Execution Proof

This appendix provides evidence of the WPScan security assessment carried out against the WordPress test environment. The screenshots show the commands executed and the main findings identified during the scan.

Figure 7.1: WPScan initial scan showing HTTP server information, XML-RPC status and directory listing.

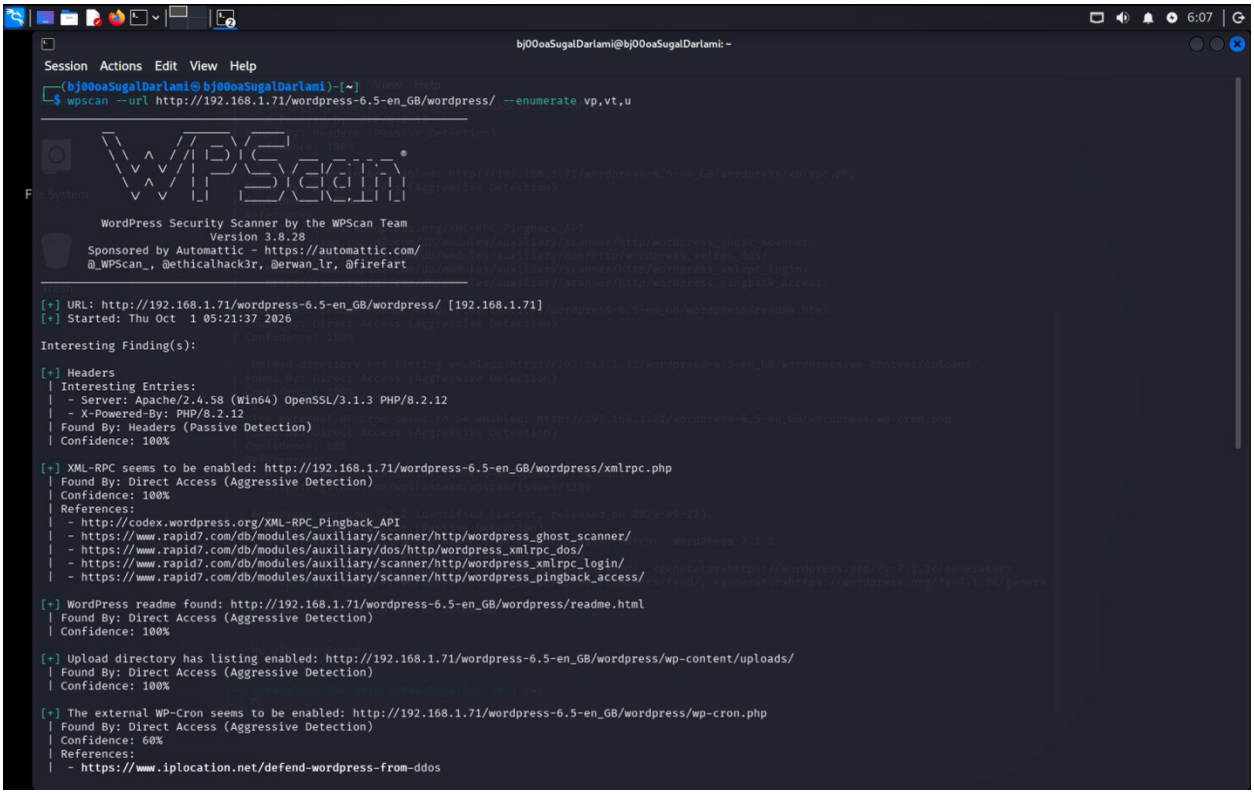


Figure 7.2: WPScan user enumeration identifying the cyber_admin account and showing scan completion details.



Raw WPScan Output

wpscan --url http://192.168.1.71/wordpress-6.5-en_GB/wordpress/ --enumerate vp,vt,u

```
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __ ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

      WordPress Security Scanner by the WPScan Team
                     Version 3.8.28
          Sponsored by Automattic - https://automattic.com/
          @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[+] URL: http://192.168.1.71/wordpress-6.5-en_GB/wordpress/ [192.168.1.71]
[+] Started: Thu Oct  1 05:21:37 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entries:
 |  - Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
 |  - X-Powered-By: PHP/8.2.12
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] XML-RPC seems to be enabled: http://192.168.1.71/wordpress-6.5-
en_GB/wordpress/xmlrpc.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
 | References:
 |  - http://codex.wordpress.org/XML-RPC_Pingback_API
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
 |                                                                                -
https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] WordPress readme found: http://192.168.1.71/wordpress-6.5-
en_GB/wordpress/readme.html
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
```

[+] Upload directory has listing enabled: http://192.168.1.71/wordpress-6.5-en_GB/wordpress/wp-content/uploads/
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://192.168.1.71/wordpress-6.5-en_GB/wordpress/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 7.1.2 identified (Latest, released on 2026-09-22).
| Found By: Meta Generator (Passive Detection)
| - http://192.168.1.71/wordpress-6.5-en_GB/wordpress/, Match: 'WordPress 7.1.2'
| Confirmed By: Rss Generator (Aggressive Detection)
| - http://192.168.1.71/wordpress-6.5-en_GB/wordpress/feed/, <generator>https://wordpress.org/?v=7.1.2</generator>
| - http://192.168.1.71/wordpress-6.5-en_GB/wordpress/comments/feed/, <generator>https://wordpress.org/?v=7.1.2</generator>

[i] The main theme could not be detected.

[+] Enumerating Vulnerable Plugins (via Passive Methods)

[i] No plugins Found.

[+] Enumerating Vulnerable Themes (via Passive and Aggressive Methods)
Checking Known Locations - Time: 00:02:47
<==========> (652 / 652) 100.00% Time: 00:02:47

[i] No themes Found.

[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:05
<==========> (10 / 10) 100.00% Time: 00:00:05

[i] User(s) Identified:

[+] cyber_admin
| Found By: Wp Json Api (Aggressive Detection)
| - http://192.168.1.71/wordpress-6.5-en_GB/wordpress/wp-json/wp/v2/users/?per_page=100&page=1

| Confirmed By:
| Rss Generator (Aggressive Detection)
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Thu Oct 1 05:24:51 2026
[+] Requests Done: 699
[+] Cached Requests: 5
[+] Data Sent: 190.417 KB
[+] Data Received: 1002.97 KB
[+] Memory used: 294.168 MB
[+] Elapsed time: 00:03:14

The following is the raw terminal output recorded during the security scan. It is included as supporting evidence for the findings discussed in the report.